

Line deletion (d and D)

The 'd' operator is used to delete all data in the pattern space. For example:

```
$ seq 4 | sed '3 d'
```

```
1
2
4
```

The D operator also performs a deletion operation, but it removes the data in the pattern space until the first `\n` character is found. For instance, let's suppose we have the data `'line1\nline2\nline3'` in the pattern space; when `D` is applied, the content of pattern space becomes `'line2\nline3'`.

Any statement that comes after a `d` or `D` in the `sed` script will not be executed; it skips the following commands, and starts the next iteration.

Hold buffer (h and H) and exchange operation (x)

The hold buffer is a special storage location, like the pattern space. You can copy the contents of the pattern space to this storage area. When 'h' is used, it replaces the current content of the hold buffer with a copy of the pattern space. When 'H' is used, a copy of the pattern space will be appended to the contents of the hold buffer.

When you need to move the contents of the hold buffer back to the pattern space to work on it, use the exchange ('x') operator, which swaps the contents of the hold buffer and the pattern space. If you swap again, the pattern space will contain the original data. Let's look at a simple example:

```
File lines.txt:
line 1
line 2
line 3
line 4
line 5
```

Now, you need to store the first line in the hold buffer, and not print it out. When `sed` reaches the end-of-file, it should print the first line instead of the last line:

```
$ sed -n '1{h; n};$x;p' lines.txt
line 2
line 3
line 4
line 1
```

In the above script, `1{h; n}` says that when the line number is 1, execute the group `{h;n}`. The group uses the copy-to-hold-buffer operator 'h' and the read-next-line operator 'n'. `$x` says that when `sed` reaches the last line (\$), exchange the hold buffer and the pattern space contents, so that the pattern

space will contain the (earlier-stored) first line. The operator 'p' is used to print the pattern space for every execution.

Get operator (g and G)

In the case of the hold buffer, when you needed to copy its contents to the pattern space, you used the exchange ('x') operator to swap the contents of the hold buffer and the pattern space. Additionally, the get operator ('g') can be used to copy the hold buffer contents to the pattern space, deleting the contents of the pattern space while doing so. The 'G' operator appends the contents of the hold buffer to the pattern space, instead of deleting the pattern space's contents and replacing it with the hold buffer's content.

Let's use the file `lines.txt` to illustrate the example:

```
$ cat lines.txt | sed -n '$p;h;n;G;p;'
line 2
line 1
line 4
line 3
line 5
```

In the above script, swap two consecutive lines, using `G` and `h`.

Operator	Effect
\$p	Prints the pattern space when the last line is reached.
H	Copies the pattern space to the hold buffer.
N	Reads the next line into the pattern space.
G	Appends the hold buffer contents to the pattern space.
P	Prints the current pattern space.

The above sequence is executed iteratively until end-of-file is reached.

Flow control with labels; and testing with 't'

You would never expect `sed` to support looping of statements. Yes, it has a branching command, 'b', which can jump to a label specified as `:labelname`. Branching can be performed by using 'b labelname'. If the label is not specified, it will branch to the end of the `sed` script. There is an additional operator called test ('t'), which is used for conditional jumps and is identical to 'b'. The only difference between the two is that in 't' the branching will occur only if the last statement is executed successfully. For branching with `b`, you may need to explicitly quit from script execution with the 'q' operator, since otherwise, it never ends the execution loop. Let's go through an example to test whether a string is in the format `A'B'`:

```
$ echo AAABBB | sed ':again; s/A(.*)B/\1/; t again; s/^/TRUE/; t; s/.*/FALSE/'
```